OBJECT ORIENTED PROGRAMMING WITH C++

ASSIGNMENT 2 – CO2

Functions and Object-Oriented Programming Concepts

Student details	Fill before submission
Name	HARIPRASAD R
Register Number	192421179
Course / Section	B.TECH IT – 3rd Year
Submission Date	11-11-2026
GitHub Repository Link	https://github.com/<username>/DSA01-<RegisterNumber>-Assignment2

Important submission note

This document provides worked explanations, source-code examples, predictions, and a validation plan. You must compile and execute every program on your own computer, capture genuine output/compiler-warning screenshots, enter your actual compiler version and timings, and commit each task separately. Do not present the sample timings or blank evidence fields as measured results.

Recommended compiler commands

```
g++ --version
g++ -std=c++17 -Wall -Wextra -pedantic task.cpp -o task
./task
```

On Windows PowerShell, run the executable as: .\task.exe. Use the exact command and compiler version shown by your own system in README.md.

Q1 – Call by Value, by Reference and by Pointer (6 Marks)

1. Three concession implementations

For a fee of Rs. 45,000 and a 10% concession, the expected revised fee is Rs. 40,500. The value version changes only its local parameter; reference and pointer versions can modify the caller's variable.

```
#include <iostream>
using namespace std;

void concessionByValue(long long fee, int percent) {
    fee -= fee * percent / 100;
    cout << "Inside value function: " << fee << '\n';
}
void concessionByReference(long long& fee, int percent) {
    fee -= fee * percent / 100;
}
void concessionByPointer(long long* fee, int percent) {
    if (fee != nullptr)
        *fee -= (*fee) * percent / 100;
}
int main() {
    long long a = 45000, b = 45000, c = 45000;
    concessionByValue(a, 10);
    concessionByReference(b, 10);
```

```
    concessionByPointer(&c, 10);
    cout << "Stored after value: " << a << '\n';
    cout << "Stored after reference: " << b << '\n';
    cout << "Stored after pointer: " << c << '\n';
}
```

Prediction and expected result

```
Inside value function: 40500
Stored after value: 45000
Stored after reference: 40500
Stored after pointer: 40500
```

The value parameter is a separate copy, so its update is lost when the function returns. A reference parameter is an alias for the caller's variable. A pointer parameter stores an address; dereferencing it changes the object at that address. The null check prevents dereferencing a null pointer.

2. Given Predict the Output program

Write this prediction in your report before compiling the supplied program:

```
11 12 13
13 13
16 13 6
```

Explanation: change(a,b,&c) increments its local copy of a to 11, the caller's b from 2 to 12, and the caller's c from 3 to 13. Therefore the first line is 1 12 13. bigger(a,c) returns a reference to the larger object; a=1 and c=13, so assigning 100 changes c. The second line is 1 100. show(1) uses y=5 and z=10, giving 16; show(1,2) uses z=10, giving 13; show(1,2,3) gives 6.

The exact output is:

```
1 12 13
1 100
16 13 6
```

The prediction above is corrected here to emphasize the exact result: the caller's a does not change in change(), and bigger(a,c) assigns 100 to c.

3. Complete the Code – TODO 1

```
void applyConcession(long long& fee, int percent)
{
    fee -= fee * percent / 100;
}
```

Expected complete program output:

```
after call by value   : 45000
after call by reference: 40500
```

The call by value leaves hostelFee at 45000 because only the copied parameter is reduced.

4. Return-a-value alternative and complexity

```
long long applyConcession(long long fee, int percent) {
    return fee - fee * percent / 100;
}
// Caller:
hostelFee = applyConcession(hostelFee, 10);
```

Parameter style	What is passed	Caller changes?	Extra parameter storage
long long by value	A copy of the integer	No, unless returned/assigned	O(1), one integer
long long&	Alias to original integer	Yes	O(1), no object copy
long long*	Address of integer	Yes, after dereference	O(1), one address
const Student&	Read-only alias to object	No modification through	O(1), no Student copy

		parameter	

For a large Student object, pass by value copies all its data (copy cost proportional to the object's size); pass by reference or const reference avoids that copy. Use const Student& when the function needs to inspect an object but must not modify it.

Design decision: use a reference when the operation is explicitly an update to the student's stored fee and no separate result is needed. Use a returned value when callers need to compare, log, or chain alternative fee calculations. For a concession policy, a pure returned-value function can be easier to test; for an office update operation, the reference version is direct. Document whether multiple concessions are applied sequentially or combined, and validate the percentage range.

Q2 – Return by Reference and Its Dangers (4 Marks)

1. Working implementation

```
#include <iostream>
using namespace std;

int& highestMark(int marks[], int size) {
    int best = 0;
    for (int i = 1; i < size; ++i)
        if (marks[i] > marks[best]) best = i;
    return marks[best];
}

// Deliberately unsafe: do not call this function.
int& badReference() {
    int local = 75;
    return local; // warning: reference to local variable returned
}

int main() {
    int marks[] = {78, 91, 84, 88};
    highestMark(marks, 4) += 2;
    cout << "Moderated highest mark: " << highestMark(marks, 4) << '\n';
}
```

Expected output: Moderated highest mark: 93

highestMark returns an alias to an array element, not a temporary copy. Therefore it can be used on the left side of an assignment, for example highestMark(marks,4)=95. The array remains alive after the function returns.

2. Dangling reference explanation

badReference returns a reference to local. The local variable's lifetime ends when the function returns. The returned reference then dangles; using it is undefined behaviour. A compiler commonly warns that the function returns a reference to a local variable. The exact wording depends on the compiler. Capture the warning from your own build; do not execute or rely on the dangling reference.

Return type	Caller can modify?	Safety / use
int&	Yes	Use only when the referred object outlives the reference
const int&	No through this reference	Avoids a copy, but lifetime must still be valid
int	Modifies only the returned copy	Safe independent value; suitable for small values

If a std::vector grows and reallocates its storage, references, pointers and iterators to its elements can be invalidated. A returned index remains an integer and can be used to access the vector again after growth, provided the index is still valid. A pointer/reference is efficient but needs a clear lifetime guarantee.

3. Index and pointer alternatives

```
int highestIndex(const int marks[], int size) {
    int best = 0;
    for (int i = 1; i < size; ++i)
        if (marks[i] > marks[best]) best = i;
    return best;
}
// Use: int i = highestIndex(marks, n); marks[i] += 2;

// Pointer alternative:
int* highestPointer(int marks[], int size) {
    int best = 0;
    for (int i = 1; i < size; ++i)
        if (marks[i] > marks[best]) best = i;
    return &marks[best];
}
// Use: *highestPointer(marks, n) += 2;
```

Finding the maximum takes O(n) time and O(1) auxiliary space in all three versions. Index is usually the clearest choice for a vector that may reallocate; reacquire the element after growth. A pointer has the same invalidation concern as a reference.

Q3 – Default Arguments and Function Overloading (6 Marks)

1. Why the original declarations are ambiguous

```
double charge(double weight);
double charge(double weight, bool express);
double charge(double weight, bool express = false, bool insured = false);
```

A call charge(2.0) can match all three declarations: the one-parameter overload, the two-parameter overload with a default for its second parameter only if that declaration has a default (it does not in the stated version), and the three-parameter declaration through its defaults. If the two-parameter overload is also given a default, it creates further overlap. The three-parameter overload overlaps the one-parameter overload for a one-argument call, so the call is ambiguous. Default arguments do not create a new function signature; they allow fewer arguments to call the same function.

2. One unambiguous redesign

```
#include <iostream>
using namespace std;

struct Parcel {
    double weightKg;
    bool express;
    bool insured;
};

double charge(double weightKg, bool express = false, bool insured = false) {
    double amount = weightKg * 40.0;
    if (express) amount += 50.0;
    if (insured) amount += 20.0;
    return amount;
}
double charge(int weightGrams) {
    return charge(weightGrams / 1000.0, false, false);
}
double charge(const Parcel& p) {
    return charge(p.weightKg, p.express, p.insured);
```

```
}

int main() {
    Parcel p{2.0, true, true};
    cout << charge(2) << '\n';       // int overload
    cout << charge(2.0) << '\n';     // double overload
    cout << charge(2.0f) << '\n';    // float promotes to double
    cout << charge(2000) << '\n';    // int overload
    cout << charge(2.0, true) << '\n';
    cout << charge(p) << '\n';       // Parcel overload
}
```

The six calls select, in order: int grams overload; double kilograms overload; double overload by float-to-double promotion; int grams overload; double overload with express=true and insured defaulted to false; Parcel overload. The illustrative pricing rule is 40 currency units per kg, plus 50 for express and 20 for insurance; change it if your assignment specifies a different tariff.

3. Overload resolution and default arguments

- Exact match is preferred over a promotion or conversion.
- A promotion (for example float to double) is generally preferred over a standard conversion such as int to double.
- If candidates remain equally good, the call may be ambiguous.
- Default arguments are supplied only after the compiler has selected a viable function.
- Defaults must be supplied from right to left: a parameter with a default cannot be followed by a parameter without a default.

4. Parcel / template alternative and complexity

A single Parcel structure groups related options and can be extended with fields such as fragile or destination. A function template can generalize numeric weight types, but validation and conversion rules must be designed carefully. Overload resolution is compile-time work and adds no runtime dispatch cost. Each overload above takes scalar values or a const reference; Parcel-by-value would copy the structure, while const Parcel& avoids that copy. Adding a new option is often clearer in a Parcel than adding more boolean parameters.

Q4 – Specifying a Class with an Array Inside It (6 Marks)

1. SavingsAccount with last 10 transactions

```
#include <iostream>
#include <string>
using namespace std;

class SavingsAccount {
private:
    string accountNumber;
    double balance;
    double transactions[10];
    int count;

    void addTransaction(double amount) { // inside class: implicitly inline
        if (count < 10) {
            transactions[count++] = amount;
        } else {
            for (int i = 1; i < 10; ++i)
                transactions[i - 1] = transactions[i];
            transactions[9] = amount;
        }
    }
```

```
public:
    SavingsAccount(const string& number, double opening)
        : accountNumber(number), balance(opening), count(0) {}

    bool deposit(double amount);
    bool withdraw(double amount);
    void miniStatement() const;
};

bool SavingsAccount::deposit(double amount) {
    if (amount <= 0) return false;
    balance += amount;
    addTransaction(amount);
    return true;
}
bool SavingsAccount::withdraw(double amount) {
    if (amount <= 0 || amount > balance) return false;
    balance -= amount;
    addTransaction(-amount);
    return true;
}
void SavingsAccount::miniStatement() const {
    cout << "Account: " << accountNumber << "\nBalance: " << balance
         << "\nRecent transactions:\n";
    for (int i = 0; i < count; ++i)
        cout << transactions[i] << '\n';
}

int main() {
    SavingsAccount a("SB1001", 1000);
    for (int i = 1; i <= 11; ++i) a.deposit(i * 10);
    a.miniStatement();
}
```

The private array prevents outside code from inserting invalid transactions, changing the count, or corrupting the balance. The public methods enforce the account rules.

2. Shift array versus circular buffer

Design	Add when full	Time per new transaction	Extra storage
Shift fixed array	Move 9 elements left, write at end	O(k), k=10 here	O(1) fixed array
Circular buffer	Overwrite oldest slot and advance start index	O(1)	O(1) fixed array plus index
std::deque	push_back, then pop_front if size exceeds limit	O(1) at ends	O(k) storage

For only 10 entries, shifting is simple and its cost is small. If the requirement becomes 50, the array capacity and loop bounds must change. A circular buffer changes only the capacity constant and keeps insertion O(1), so it is the stronger design if the transaction limit may grow or insertions are frequent.

3. Circular-buffer alternative pseudocode

```
capacity = 10
start = 0
count = 0
add(value):
    if count < capacity:
        index = (start + count) % capacity
        count = count + 1
    else:
        index = start
        start = (start + 1) % capacity
    data[index] = value
```

```
printOldestToNewest:
    for i = 0 to count-1:
        print data[(start+i) % capacity]
```

A deque is convenient when dynamic capacity is desired; a fixed circular buffer is predictable and avoids shifting. Both retain O(k) space for k saved transactions.

Q5 – Static Members and Arrays of Objects (7 Marks)

1. Static counter and array of tickets

```
#include <iostream>
#include <string>
using namespace std;

class BusTicket {
    static int issued;
    static const int CAPACITY = 3;
    int seat;
    string name;

public:
    explicit BusTicket(const string& n) : seat(0), name(n) {
        if (issued < CAPACITY) {
            ++issued;
            seat = issued;
        }
    }
    static int count() { return issued; }
    void print() const {
        cout << name << ": "
             << (seat ? "Seat " + to_string(seat) : string("Waiting"))
             << '\n';
    }
};
int BusTicket::issued = 0;

int main() {
    BusTicket trip[4] = {
        BusTicket("Asha"), BusTicket("Bala"),
        BusTicket("Chitra"), BusTicket("Dev")
    };
    for (const BusTicket& t : trip) t.print();
    cout << "Issued: " << BusTicket::count() << '\n';
}
```

Expected output:

```
Asha: Seat 1
Bala: Seat 2
Chitra: Seat 3
Dev: Waiting
Issued: 3
```

issued is static because it belongs to the class and is shared by every BusTicket object. The declaration appears inside the class, while the definition `int BusTicket::issued = 0;` appears once outside the class. A static member function has no this pointer because it is not called for one particular object; therefore it cannot directly access non-static fields such as seat or name.

2. Given Predict the Output

Prediction before execution:

```
101 103 104 4
```

The static next starts at 100 because of the out-of-class definition. Three array objects are constructed, receiving IDs 101, 102 and 103. The extra object receives 104. issued() returns next-100, which is 4. The constructor runs four times.

3. Complete the Code – TODO 1 and TODO 2

```
// TODO 1, inside BusTicket constructor:
if (issued < CAPACITY) {
    ++issued;
    seat = issued;
}

// TODO 2, inside count():
return issued;
```

The completed program output is the five-line expected output shown above.

4. Function called twice and alternatives

```
void createTrip() {
    BusTicket local[2] = {BusTicket("Esha"), BusTicket("Farah")};
    for (const BusTicket& t : local) t.print();
    cout << "Count now: " << BusTicket::count() << '\n';
}
// Call createTrip(); createTrip();
```

The static count persists after the local array is destroyed. It is not reset when a function returns. With capacity 3, the first call issues two seats; the second call can issue only the remaining seat, and its second ticket is waiting. Later calls continue to see the shared count.

Counter design	Encapsulation	Two buses at once
Static class member	Kept inside BusTicket but shared by all tickets	Not naturally independent: both buses share one count
Global variable	Global and accessible widely; weak encapsulation	Possible, but separate globals are awkward
TicketOffice object	Counter belongs to a specific office/bus object	Best: create one office per bus and pass it to tickets

All counter updates are O(1) time and O(1) counter space. If two buses operate independently, a TicketOffice object is the most flexible design because each bus can own its own capacity and issued count.

Q6 – Friend Functions and Inline Functions (8 Marks)

1. Completed friend functions

```
#include <iostream>
#include <cstdlib>
using namespace std;

class Time {
    int h, m;
public:
    Time(int hh, int mm) : h(hh), m(mm) {}
    friend int minutesBetween(const Time& a, const Time& b);
    friend bool earlier(const Time& a, const Time& b);
};

int minutesBetween(const Time& a, const Time& b) {
    int first = a.h * 60 + a.m;
    int second = b.h * 60 + b.m;
    return abs(first - second);
}
bool earlier(const Time& a, const Time& b) {
```

```
    return a.h * 60 + a.m < b.h * 60 + b.m;
}
int main() {
    Time departure(9, 45), arrival(13, 10);
    cout << minutesBetween(departure, arrival) << ' '
         << boolalpha << earlier(arrival, departure) << '\n';
}
```

Expected output: 205 false. 09:45 is 585 minutes after midnight; 13:10 is 790 minutes after midnight; the difference is 205. arrival is not earlier than departure.

2. Correctness tests

```
Time morning(11, 50), afternoon(12, 10), same(12, 10);
cout << minutesBetween(morning, afternoon) << ' '
     << earlier(morning, afternoon) << '\n'; // 20 true
cout << minutesBetween(same, afternoon) << ' '
     << earlier(same, afternoon) << '\n';    // 0 false
```

The constructor assumes valid 24-hour values ($0 \le h \le 23$ and $0 \le m \le 59$). A production class should validate these values. The simple minutesBetween calculation measures same-day clock difference; overnight journeys need date/day handling.

3. Friend, member and getter alternatives

- A friend is a non-member function declared inside the class with the friend keyword. It is called as minutesBetween(a,b), can access private members of both objects, and is useful for symmetric operations.
- A member function is called through an object, for example a.minutesTo(b). It has an implicit this pointer for the left-hand object.
- Getters keep fields private but expose selected read-only values. The comparison can then be implemented as an ordinary non-friend function.
- Making h and m public would weaken encapsulation: any caller could set invalid times such as 99:99.

4. Inline and measurement plan

A function defined inside a class is implicitly inline. The inline keyword is a request/permission for inline substitution and, more importantly, allows suitable definitions in multiple translation units; it does not force the compiler to replace every call. The optimizer may inline functions without the keyword.

```
// Benchmark plan: use a loop with a checksum so the work is not discarded.
volatile long long checksum = 0;
for (int i = 0; i < 10000000; ++i) {
    checksum += minutesBetween(t1, t2);
}
```

Build	Command	What to record
No optimization	g++ -std=c++17 -O0 -Wall -Wextra benchmark.cpp -o bench_O0	Elapsed time from repeated runs
Optimization	g++ -std=c++17 -O2 -Wall -Wextra benchmark.cpp -o bench_O2	Elapsed time from repeated runs

Record at least five runs per build and report the median. Use the same machine, compiler, input, loop count and benchmark code. Do not invent timings: enter your measured values below.

Build	Run 1	Run 2	Run 3	Run 4	Run 5	Median
-O0						
-O2						

Each call performs a fixed number of arithmetic operations: O(1) time and O(1) extra space. At -O2, the optimizer may inline and simplify the call; at -O0, function-call overhead may be more visible. For such a tiny operation, benchmark noise and compiler decisions can dominate.

5. Deployment decision

Keep friend functions if the operation is naturally symmetric and needs direct access to both objects without exposing getters. Prefer member functions when the operation conceptually belongs to the left-hand Time object. Getters are a good compromise when the class should expose only read-only time values. The final choice should preserve private invariants; performance should be decided from repeatable measurements rather than assuming friend or inline is automatically faster.

Q7 – Day-4 Design Challenge: Diagnose, Redesign and Validate (13 Marks)

Selected program: Q4 SavingsAccount transaction history

This example is selected because the original shifting-array implementation becomes less efficient as the retained transaction count grows. When the history is full, every new transaction shifts k−1 values before storing the new value.

A. Diagnose the limitation – 2 marks

For a full history of k entries, the shifting design performs k−1 assignments for every insertion. Its insertion cost is O(k). At k=10 this is small, but at k=50,000 it can perform 49,999 moves for each new transaction. The limitation is the physical shifting of all retained entries, not the fixed array itself.

B. Measurable design objective – 2 marks

Objective: preserve the most recent k transactions in chronological order while reducing the full-buffer insertion operation from O(k) element moves to O(1) writes and one index update. Validate that both implementations retain exactly the same final k transactions after the same input sequence.

C. Revised design – 4 marks

Use a fixed-size circular buffer. Keep an array, a start index pointing to the oldest retained item, and a count. If the buffer is not full, write at (start+count)%capacity and increase count. If full, overwrite data[start] and advance start modulo capacity. This uses one additional integer index and does not allocate memory dynamically.

```
#include <iostream>
#include <vector>
using namespace std;

class CircularHistory {
    vector<double> data;
    size_t start = 0;
    size_t count = 0;
public:
    explicit CircularHistory(size_t capacity) : data(capacity) {}
    void add(double value) {
        if (data.empty()) return;
        if (count < data.size()) {
            data[(start + count) % data.size()] = value;
            ++count;
        } else {
            data[start] = value;
            start = (start + 1) % data.size();
        }
    }
    void print() const {
        for (size_t i = 0; i < count; ++i)
            cout << data[(start + i) % data.size()] << ' ';
        cout << '\n';
```

```
    }
    size_t size() const { return count; }
};

int main() {
    CircularHistory history(5);
    for (int i = 1; i <= 8; ++i) history.add(i * 10);
    history.print(); // most recent five: 40 50 60 70 80
}
```

Pseudocode / class design:

```
CircularHistory
  data[capacity]
  start = 0          // oldest element
  count = 0
  add(x):
    if count < capacity: data[(start+count)%capacity] = x; count++
    else: data[start] = x; start = (start+1)%capacity
  print(): output data[(start+i)%capacity], i=0..count-1
```

Trade-off: the circular buffer needs modulo/index logic when printing, while the shifting array stores values in chronological order directly. Both use O(k) memory.

D. Predict before implementing – 1 mark

Prediction: for capacity 5 and inputs 10,20,30,40,50,60,70,80, the final chronological output will be 40 50 60 70 80. After the buffer is full, each insertion will perform one data write and one start-index update, independent of k.

E. Implement and validate – 2 marks

Test	Shifting array expected	Circular buffer expected
Capacity 5; add 10,20,30	10 20 30	10 20 30
Then add 40,50	10 20 30 40 50	10 20 30 40 50
Then add 60,70,80	40 50 60 70 80	40 50 60 70 80
Number of retained values	5	5

For a fair timing comparison, implement both with the same capacity and feed the same generated values. Time only the insertion loop using std::chrono, repeat multiple times, and report medians. Use large capacities (for example 10,000 and 100,000) and enough insertions to fill and overwrite the buffer. Include the actual output and timing from your own computer:

Capacity / input count	Shifting median (ms)	Circular median (ms)	Output verified?
10,000 / 100,000			
100,000 / 200,000			

F. Defend the final design – 2 marks

The circular buffer meets the objective if the measured insertion time grows much more slowly as capacity increases and the retained output matches the shifting implementation. It adds index-management complexity and makes ordered printing require modular indexing. For a very small history where clarity matters more than insertion cost, shifting may remain preferable. For a fixed-size, frequently updated history, the circular buffer is the better design.

Complexity comparison

Operation	Shifting fixed array	Circular buffer
Insert while not full	O(1)	O(1)
Insert when full	O(k)	O(1)
Print k entries	O(k)	O(k)
Storage	O(k)	O(k)
Extra index state	O(1)	O(1)

Final Submission Checklist

- ☐Create repository: DSA01-<RegisterNumber>-Assignment2.
- ☐Create Task01 through Task07 folders.
- ☐Save every source file and any required input file in the correct task folder.
- ☐Run every program, including all completed TODO programs.
- ☐Write predictions before execution and preserve your original prediction in the report.
- ☐Capture genuine terminal screenshots showing compile command, output and compiler warnings where relevant.
- ☐Record compiler name/version and exact commands in README.md.
- ☐Measure Q6 timings at -O0 and -O2; measure Q7 original and redesigned versions using the same tests.
- ☐Make at least one meaningful Git commit per task; do not upload everything in one final commit.
- ☐Add your working GitHub URL to the report cover and assignment submission.
- ☐Be ready to explain reference lifetime, overload selection, static storage, friend access, circular-buffer indexing and complexity during viva.

Suggested README.md template

```
# DSA01-<RegisterNumber>-Assignment2

- Name: <Your name>
- Register Number: <Your register number>
- Compiler: <compiler name and version>
- Standard: C++17

## Compile
g++ -std=c++17 -Wall -Wextra -pedantic main.cpp -o main

## Run
Windows PowerShell: .\main.exe
Linux/macOS: ./main

## Tasks
- Task01: call by value, reference and pointer
- Task02: returning references and lifetime safety
- Task03: default arguments and overloading
- Task04: class with transaction array
- Task05: static members and object arrays
- Task06: friend and inline functions
- Task07: diagnose, redesign and validate

For each task, describe the test input, actual output, and result.
```

Academic integrity and evidence: The sample programs and explanations are study material. Execute them yourself, understand each line, add your own screenshots and measurements, and follow your faculty's rules for individual work.